

Paradygmaty Programowania.

Laboratorium nr 3 (Programowanie współbieżne.)

Michał Obara, Grzegorz Nieradka

Warszawa, Maj 2026

Zadanie 1. (2 pkt)

Pobrać plik `race_condition.py` , zapoznać się z jego treścią, następnie komentując odpowiednie linie w funkcji `main()` uruchomić skrypt i odpowiedzieć na poniższe pytania:

1. Część 1: Wyścig (wynik błędny)
 - (a) Jaka była utrata wartości zmiennej `counter` w czasie inkrementacji dla 2, 4, 8 i 16 wątków? Wyniki można przedstawić w tabeli 🐱
 - (b) Dlaczego wraz ze wzrostem liczby wątków rośnie liczba o którą zmienna `counter` jest mniejsza od oczekiwanej?
 - (c) Naturalnym rozwiązaniem zapisu inkrementacji licznika jest `counter += 1`. W skrypcie zastosowano funkcję `get_one()`.
Proszę zamienić to rozwiązanie na bardziej naturalne, czyli zamiast funkcji `get_one()` zastosować inkrementację `counter += 1` i sprawdzić wynik. Następnie skomentować otrzymany wynik.
Do tego komentarza można na szybko przeczytać tę [dyskusję o GIL-u w Pythonie na Reddicie](#).
 2. Część 2: blokada wewnątrz pętli (poprawny, równoległy)
 - (a) Czy otrzymywane wyniki są poprawne? Ile wynosił błąd dla 8 wątków?
 - (b) Jak działa `with lock:` wewnątrz pętli? Co chroni?
 - (c) Czy wątki działają równoległe? Jak to widać po czasie wykonania?
 - (d) Porównaj czasy działania między częścią 1 (wyścig) z częścią 2 (blokada wewnątrz). O ile procent wolniejsza jest wersja z blokadą?
 3. Część 3: Blokada przed pętlą (poprawna, ale serializacja)
 - (a) Czy otrzymywane wyniki są poprawne? Jednak dlaczego jest to zły wzorzec blokady?
 - (b) Dlaczego czasy wykonania rosną prawie liniowo z liczbą wątków?
 - (c) Co to znaczy, że jest "zserializowany"? Który wątek działa, gdy inne czekają?
 4. Pytania końcowe
 - (a) Który wzorzec (2 czy 3) jest lepszy i dlaczego? Spróbuj uzasadnić.
 - (b) W jakiej sytuacji blokada przed pętlą mogłaby mieć sens?
-

Prawo Amdahla (przypomnienie)

$$S = \frac{1}{(1 - P) + \frac{P}{N}} \quad (1)$$

gdzie:

S – przyspieszenie (*ang. speedup*)

P – część zrównoleglona (0 ... 1, np. 0.95 = 95% kodu może być równoległe)

N – liczba jednostek obliczeniowych (rdzeni, wątków)

Wniosek: Jeżeli $P = 0.9$ (90% kodu równoległe), nawet przy nieskończeniu wielu rdzeniach przyspieszenie $\leq 10\times$

Zadanie 2. (2 pkt)

Pobrać plik `amdahl.py` , zapoznać się z jego treścią. Następnie komentując odpowiednie linie w funkcji `main()` uruchomić skrypt i odpowiedzieć na poniższe pytania:

1. Uruchomić po kolei wszystkie trzy eksperymenty. Porównać wyniki dla 16 wątków. O ile wzrosło przyspieszenie przy przejściu z 70% na 95% części równoległej?
 2. Dlaczego przy 50% zrównoleglenia dodawanie kolejnych wątków (np. powyżej 4) daje coraz mniejszy zysk?
 3. Załóżmy, że mamy nieskończoną liczbę procesorów. Oblicz teoretyczne maksymalne przyspieszenie dla `parallel_fraction = 0.70`. Porównaj wynik z kolumną "Teoria" w programie dla 16 wątków. Jak blisko limitu już jesteśmy?
 4. Wyobraź sobie, że jesteś inżynierem i masz budżet na jedną zmianę: albo dokupienie drugiego procesora (zwiększenie liczby wątków z 16 na 32), albo optymalizację kodu tak, by część sekwencyjna spadła z 10% do 5%. Korzystając z funkcji `test_amdahl`, sprawdź, która decyzja bardziej skróci czas wykonania programu.
-

Zadanie 3. (2 pkt)

Pobrać plik `procesy.py` , zapoznać się z jego treścią następnie komentując odpowiednie linie w funkcji `main()` uruchomić skrypt i odpowiedzieć na poniższe pytania:

1. Eksperyment 1: Wątki
 - (a) Jaki był wynik eksperymentu z wątkami? Ile wynosiła zmienna `counter`?
 - (b) Czy wątki widziały nawzajem swoje zmiany?
 2. Eksperyment 2: Procesy
 - (a) Jaki był wynik eksperymentu z procesami? Dlaczego `counter` pozostał 0?
 - (b) Co to znaczy, że procesy mają "własną przestrzeń pamięci"?
 - (c) Czy procesy mogą komunikować się ze sobą? Jeśli tak to jak?
 3. Eksperyment 3: Wydajność
 - (a) Co było szybsze - tworzenie wątków czy procesów? O ile razy?
 - (b) Dlaczego wątki mają mniejszy narzut od procesów?
-